PSI DISC · ASSOCIATE DATA ENGINEER ASSESSMENT

DHIS2 Health Data Pipeline.

Technical Walkthrough — Julius Nyambok

PySpark | Star Schema | 4 Bonus Challenges

PRESENTER
JULIUS NYAMBOK

SCOPE
7 TASKS · 4 BONUS · SINGLE COMMAND

THROUGHPUT
152K ROWS · 5 COUNTRIES · 137S

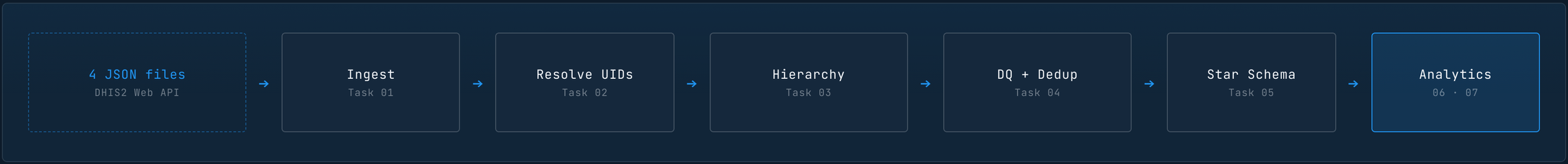
DATE
MAY 2026

SECTION 00 · PIPELINE OVERVIEW

Seven tasks, four bonuses, one command.

152K+	5	~137s	local[*]
DATA VALUES	COUNTRIES	END-TO-END	PYSPARK MODE

- **7 core tasks + 4 bonus challenges** — single-command execution, no orchestration glue required
- PySpark `local[*]`, processes **152,000+ data values** across 5 countries
- Exit codes: `0` success · `1` critical DQ failure · `2` runtime error



TASK 01 · JSON INGESTION

Explicit schemas. Never inferSchema.

- Loads **4 separate DHIS2 Web API files**: metadata, org_units, programs, data_values
- Explicit `StructType` schemas applied at DataFrame creation — `json.load()` gives row-level quarantine control **before** any DataFrame exists
- `value` column kept as `StringType` to preserve the **0 vs NULL** distinction downstream
- Quarantines rows missing `orgUnit`, `period`, or `dataElement`, or with invalid period format
- **Quarantine rate > 10%** → pipeline exits with code `1`

metadata.json	org_units.json	programs.json	data_values.json
40 DEs · 21 COCs	658 units · 576 facilities	16 programs · 5 areas	152,802 rows · 43 MB

INGESTION · PYTHON — INGEST.PY

```python
# 1 · load 4 files separately with json.load()
with open("data_values.json") as f:
    raw = json.load(f)["dataValues"]

# 2 · quarantine rows missing required fields
for row in raw:
    if not row.get("orgUnit") or not row.get("period"):
        quarantine.append({..., "reason": "missing_field"})
    else:
        valid.append({..., "value": str(row.get("value"))})

# 3 · build DataFrame with explicit schema
df = spark.createDataFrame(valid, schema=_DV_SCHEMA)

# 4 · quarantine gate
quarantine_rate = bad_rows / total_rows
if quarantine_rate > 0.10:
    sys.exit(1)  # critical DQ failure
```

TASKS 02 + 03 · UID RESOLUTION & HIERARCHY

Opaque UIDs become human geography.

- DHIS2 stores everything as opaque **11-character UIDs** — joins, not lookups
- **Task 02:** `F.broadcast()` on metadata lookups — 40 DEs, 3 COCs fit comfortably in driver memory
- `left_anti` join isolates **ghost DE UIDs** → removed. Orphaned COCs → flagged `is_orphaned_coc=True`, kept
- **Task 03:** split the `path` column on `/`, extract ancestor UIDs at positions 1–3
- Self-join the org unit table **3 additional times** to resolve country, region, district by name
- **No hardcoded UIDs** anywhere in the pipeline

HIERARCHY RESOLUTION EXAMPLE — PATH → NAMES

`/OU_KE/OU_KE_R1/OU_KE_D1/OU_KE_F1`

↓ split · self-join × 3 ↓

L1 · COUNTRY	L2 · REGION	L3 · DISTRICT	L4 · FACILITY
Kenya	Nairobi Region	Nairobi North	Kenyatta National Hospital

```
# left-anti isolates ghost UIDs (in fact, not in metadata) ghosts = data_values.join(
F.broadcast(metadata_des), "dataElement", "left_anti", )
```

TASK 04 · 0 VS NULL

The most important design decision.

Collapsing zero into missing — or missing into zero — produces misleading programme dashboards. We keep them strictly separate.

VALUE = 0

0

Facility is operational, tested patients, no cases found.

An actionable signal — a reported zero. Programme staff can act on it: stockouts averted, surveillance working as expected.

```
is_explicit_zero  =  value == "0"
```

VS

VALUE = NULL

∅

Facility did not report. Status is unknown.

Could be a closed facility, a system failure, a missing form, a stockout. Not a zero — it is the absence of a signal.

```
is_missing_value  =  value IS NULL OR value = ""
```

Two separate boolean flags. Never merged. Downstream consumers — reporting rate, anomaly detection, top-5 underreporters — choose how to handle each case explicitly.

TASK 04 · DEDUPLICATION

Exact, near-duplicate, late. All handled.

- **Exact duplicates:** `dropDuplicates()` on all business-key columns
- **Near-duplicates:** same composite key (`dataElement + period + orgUnit + COC`), different value → `Window` partitioned by composite key, ordered by `lastUpdated DESC` , keep `row_number == 1` → **most recent correction wins**
- `typed_value` : cast `STRING value` to `DoubleType` after dedup — original string preserved for audit
- **Late reporting flag:** `lastUpdated > 60 days` after period end → `is_late_reported = True`

NEAR-DUPLICATE RESOLUTION · PYTHON — DEDUP.PY

```python
from pyspark.sql import Window, functions as F

composite_key = [
    "dataElement", "period",
    "orgUnit",     "categoryOptionCombo",
]

# 1 · most-recent-correction-wins
w = (Window
       .partitionBy(*composite_key)
       .orderBy(F.col("lastUpdated").desc()))

deduped = (df
  .withColumn("rn", F.row_number().over(w))
  .filter("rn = 1")
  .drop("rn"))

# 2 · cast for analytics; keep raw for audit
deduped = deduped.withColumn(
    "typed_value", F.col("value").cast("double"),
)

# 3 · late-reporting flag (> 60 days post period-end)
deduped = deduped.withColumn(
    "is_late_reported",
    F.datediff("lastUpdated", "period_end") > 60,
)
```

TASK 05 · STAR SCHEMA

Dimensional model, natural keys, partitioned for the query.

- **4 dimension tables**: `dim_data_element`, `dim_org_unit`, `dim_period`, `dim_program`
- `fact_service_delivery` partitioned by `health_area` then `year_month`
- Partition matches the query pattern: **programme managers filter by health area first, then by time**
- Stores both raw `STRING value` and `typed_value DOUBLE` — raw for audit, typed for aggregation
- **No surrogate keys** — DHIS2 UIDs are globally unique, stable, natural keys

TASK 06 · ANALYTICS & WINDOW FUNCTIONS

Catalyst-optimised. Zero Python UDFs.

A1 · TREND

Month-over-month % change

Per indicator, per facility. Computes signed percentage change against the previous period using **F.lag()** over a facility/indicator window.

```
F.lag("typed_value").over(w_facility_indicator)
```

A2 · SMOOTHING

3-month rolling average

Per facility, per indicator. Window with **rowsBetween(-2, 0)** for a trailing 3-period mean — robust to single-period anomalies.

```
w.rowsBetween(-2, 0) · F.avg("typed_value")
```

A3 · COVERAGE

Country reporting rate

Reporting facilities / total known facilities from the org unit hierarchy. Denominator includes facilities that *never* reported — silent failure visible.

```
countDistinct(reported_orgUnit) / total_facilities
```

A4 · TRIAGE

Top-5 underreporters per health area

Ranked by **periods with zero *or* missing data** — not by total value. Catches both true low performers and silent dropouts.

```
F.rank().over(w_area.orderBy(missing_periods.desc()))
```

Zero Python UDFs. All native Spark SQL functions — execution stays in the JVM, Catalyst optimises the entire plan, and `collect_list` never materialises a partition into the driver.

BONUS · B1 – B4

Four bonus challenges, all shipped.

B1

Data Contract

YAML schema validates columns, types, nullability, and value ranges before any downstream work. Pipeline exits with code `1` on violation — no silent corruption.

CONTRACT.YAML · PRE-FLIGHT

B2

Incremental Load

The `--incremental` flag inspects existing Parquet partitions and processes **only new periods**. No reprocessing, no double-counting, no manual coordination.

--INCREMENTAL · PARTITION-AWARE

B3

Anomaly Detection

Z-score > 3 against 12-month rolling stats per facility / indicator. Also flags **period spike > 300% MoM**. Output to CSV for programme review.

Z > 3 · MOM > 300% · CSV

B4

Metadata Drift

Diffs the current `metadata.json` against a stored snapshot. Reports **added · removed · renamed** data elements so downstream dashboards never silently break.

SNAPSHOT DIFF · AUDIT LOG

SECTION 08 · PIPELINE RELIABILITY

Predictable failure, observable success.

0

EXIT · SUCCESS

All tasks passed. Fact rows written, contract satisfied, summary logged.

1

EXIT · DQ FAILURE

Quarantine > 10%, contract violation, or zero fact rows. **Hard stop.**

2

EXIT · RUNTIME ERROR

Unhandled exception. Stack trace + structured log captured for operator triage.

Structured logging

EVERY TASK

Every task logs **start time, row counts, and elapsed time**. End-of-run pipeline summary captures bytes read, partitions written, and DQ flag counts — pipe into any log aggregator without parsing.

```
[INFO] task=04_dedup rows_in=152,802 rows_out=126,681 elapsed=18.4s
```

Checkpointing

AFTER TASK 04

Writes cleaned data to **Parquet**, reads it back, breaks the Spark DAG lineage. Prevents **OOM from full-DAG materialisation** in memory-constrained environments — the failure mode that bites most local-mode runs.

```
df.write.parquet(checkpoint_path) · df = spark.read.parquet(...)
```

PRODUCTION RECOMMENDATIONS

What this becomes at 40+ country scale.

01
ORCHESTRATION
Prefect / Airflow
Scheduling, retries with exponential backoff, Slack alerts on failure.

02
STORAGE
Delta Lake / Iceberg
ACID transactions, time travel, schema evolution without rewrites.

03
COMPUTE
Databricks / EMR
Elastic Spark clusters for 40+ country scale; spot-instance pools for cost.

04
DATA QUALITY
Great Expectations
Statistical profiling, historical drift tracking, Slack-integrated DQ alerts.

05
CONTINUOUS INTEGRATION
GitHub Actions CI
pytest on every PR, schema diff checks against main, contract regression suite blocks bad merges before they ship.

SUMMARY & CLOSE

What we built. What it scales to.

152K+	5	576	40	12
DATA VALUES	COUNTRIES	FACILITIES	INDICATORS	PERIODS

01 Resolves **opaque DHIS2 UIDs**, builds the 4-level org hierarchy, preserves the **0 vs NULL** distinction, and deduplicates with most-recent-correction-wins semantics.

02 Star schema partitioned by **health_area + year_month** — matches how programme managers actually query.

03 Four bonus features delivered: **contract validation, incremental load, anomaly detection, metadata drift**.

04 All native PySpark — **no UDFs, no pandas**, Catalyst-optimised throughout. Stays in the JVM end-to-end.

BOTTOM LINE

Ready to scale to 40 countries with a cluster swap.